PAGECRAFT · ASSIGNMENT WRITE-UP

Approach and writing model

How Pagecraft turns a brand and ten topics into ten hosted, on-brand blog pages

25 October 2026 · Repository: github.com/M0tracno/FlocCare · Demo: demo/submission-walkthrough.mp4

> **In short:** a small Node.js web app takes a company name, logo, brand colour and up to 10 topics. It asks Google's free **Gemini 3.5 Flash-Lite** model for one ~1,000-word article per topic, checks each article's length, and publishes every article on its own page using one shared branded template. Projects are stored separately, so the same platform serves any number of brands.

---

1. Approach

No outside packages. The server (`server.mjs`) uses only what ships with Node.js 22. The interface is a single page (`public/index.html`), and each project is saved as a JSON file. With nothing extra to install, there is less to set up and less to break.

What happens when someone clicks “Generate”

1. **Input.** The form takes the company name, logo (PNG, JPG, WebP or GIF, under 2 MB), a six-digit hex colour and up to 10 unique topics. A live preview shows the brand before anything is generated. The browser checks the inputs, and the server checks them again because browser checks can be bypassed.
2. **Writing.** The server writes the articles one topic at a time. Each draft's words are counted and must pass a length check before it is accepted (section 4).
3. **Saving.** The finished collection is written to a temporary file that then replaces the real one in a single step, so a crash cannot leave a half-written project. Projects never overwrite each other, which is what lets one platform serve many companies.
4. **Publishing.** Every page uses one shared template filled with the company's logo and colour, which keeps all pages in a collection identical in layout. Each page lives at its own permanent address, `/p/<project-id>/<topic-slug>`, and shows its word count.
5. **Safety.** The model's Markdown is turned into HTML by a small converter written for this app. It neutralises any HTML in the text, so even a bad model reply cannot inject code into a page.

2. The writing model

Main model: Google `gemini-3.5-flash-lite`, called through the Gemini API's free tier (temperature 0.7). It was chosen after trying the alternatives live:

	Gemini 3.5 Flash-Lite	OpenRouter free models
Cost	Free	Free (50 requests per day)
Speed	6–8 seconds per article	Several minutes per article
Reliability	Answered consistently	Gemma 4 was rate-limited on every try
Length	Close to 1,000 words	Often well short

- `gemini-2.5-flash-lite` is retired for new API keys.
- `gemini-flash-latest` spent its whole output budget on internal thinking and stopped at about 250 words.

Backup: OpenRouter's free models. If Gemini returns an error and an OpenRouter key is configured, the app retries with free OpenRouter models in this order: Gemma 4 31B, Nemotron 3 Ultra, Laguna S 2.1. When a user pastes their own key into the form, keys starting with `sk-or-` go to OpenRouter and all others go to Gemini.

3. How the prompt works

Length comes from structure, not a number

Asking for “1,000 words” produced about 600. The prompt therefore describes a layout: a two-paragraph introduction (~100 words), six headed sections of three paragraphs each (~140 words per section), and a conclusion (~80 words). That adds up to roughly 1,000 words, and the model follows a layout far more reliably than a count.

Content rules

- Write original text.
- Do not invent citations, statistics, clinical evidence, product claims or testimonials.
- On health topics, give general educational information only, with no diagnosis or treatment advice, and suggest professional review.
- Do not mention being an AI or add a title (the page supplies it).

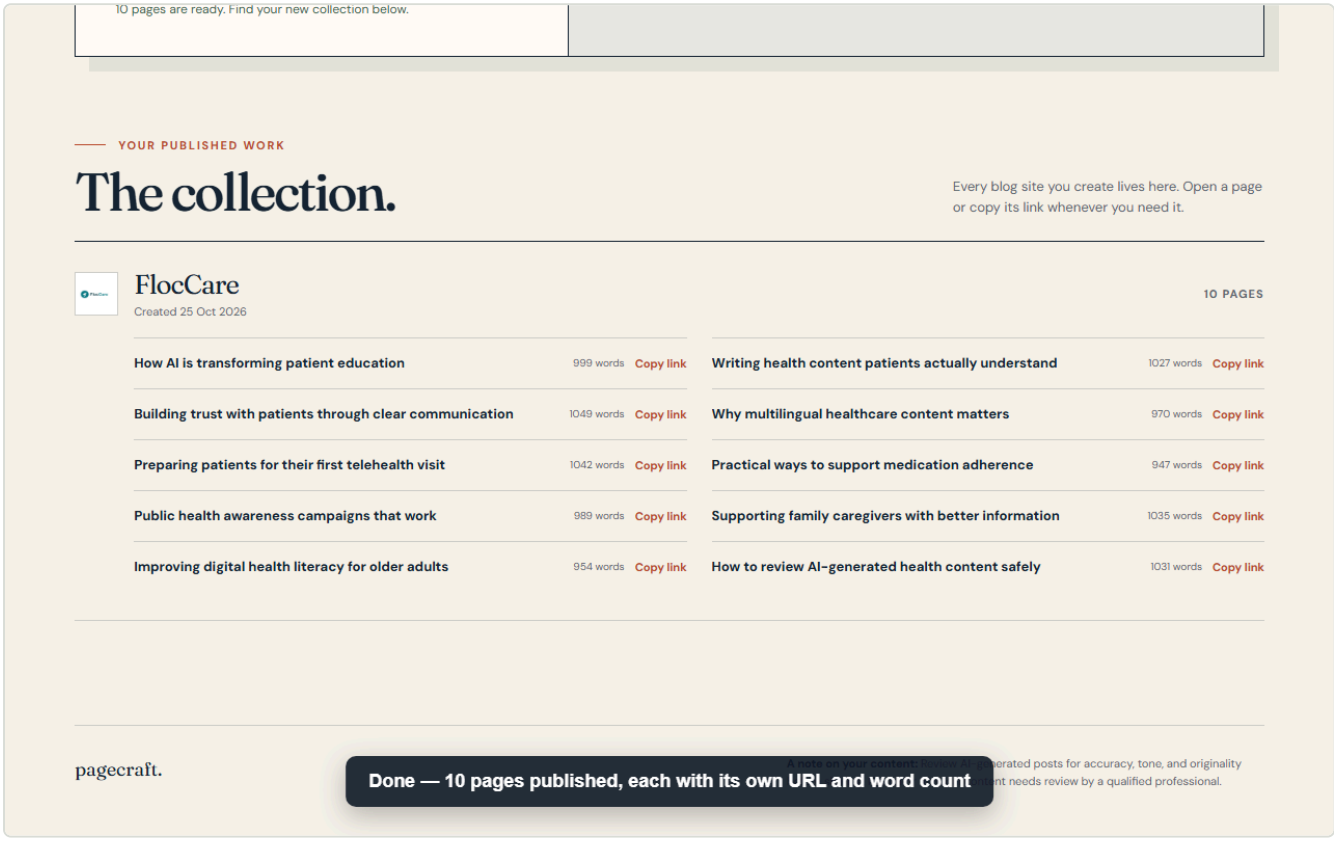
4. The length check

The target is 950–1050 words, and an article is never published outside 900–1100. Models are poor at hitting a count when asked to revise, so the server counts the words itself and steers the model:

- **Revise from the best draft.** Each revision starts from the draft closest to 1,000 words so far, so one bad revision cannot lose a near miss.
- **Ask for adjusted amounts.** Testing showed the model adds only about half the words requested and cuts too many. A short draft therefore asks for twice the missing words, and a long draft asks to cut three-quarters of the excess.

- **Start over when revising stops helping.** If a revision is not closer than the best draft, the next attempt is a fresh draft. This matters because the model sometimes returned the same draft unchanged several times.
- **Up to 6 attempts.** If none land in 950–1050, the closest draft is published only if it is within ±10% of 1,000 words. Otherwise the run fails rather than publishing an article of the wrong length.
- **Cut-off drafts do not count.** A draft that hit the model's output limit is treated as a failed attempt.
- **Rate limits.** If the API reports it is busy (errors 429 or 503), the server waits 20 seconds and tries again, up to twice.

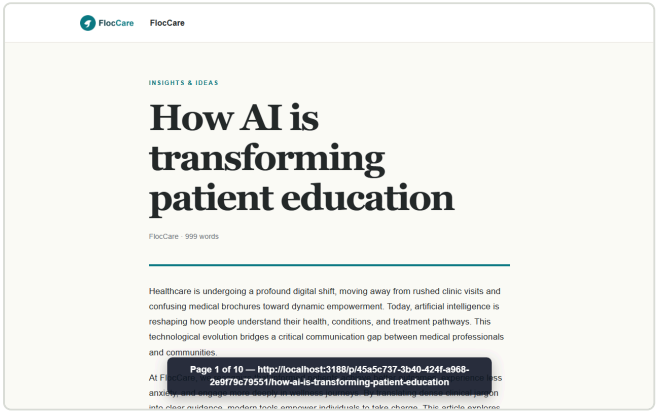
Result in the recorded demo: all 10 FlocCare articles came out between 947 and 1,049 words, most on the first or second attempt.



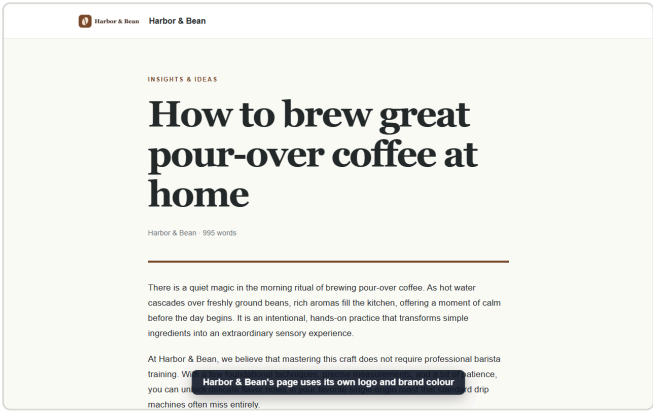
The collection after the live run: ten FlocCare pages, each with its own link and word count.

5. How the brief is met

Requirement	How Pagecraft meets it
10 separate pages	One page per topic, each at its own permanent URL.
~1,000-word posts	Structured prompt plus the length check; 947–1,049 words in the demo.
Consistent layout	One shared template applies the uploaded logo and brand colour to every page.
Pages hosted	The app serves every page and lists them with copy-link buttons; `render.yaml` is included for a public deployment.
Multiple use cases	Each brand is a separate project. The demo adds Harbor & Bean alongside FlocCare, each keeping its own branding.



FlocCare page: logo, teal brand colour, generated article.



Same template, second brand: Harbor & Bean in brown.

6. How it was verified

- **Automated test** (`tests/integration.test.mjs`). Using a fake model service, it checks that 10 pages are published within the word range and carry the brand; that projects persist and a second project sits alongside the first; that bad input is rejected; that keys are routed to the right provider and the switch from Gemini to OpenRouter works; and that the API key never appears in the API's output.
- **Live demo** (`demo/submission-walkthrough.mp4`). Real Gemini output, no mock: FlocCare with 10 topics, then Harbor & Bean on the same platform.

7. Limits

- **No logins.** Everyone who opens the site sees every project.
- **Hosting.** For the demo, pages are served by the app on a local machine. A public URL needs a deployment (configuration included).
- **Free-tier limits.** Free plans cap daily generation and may keep a copy of the prompts sent.
- **Review needed.** These are AI drafts. Health content in particular needs fact checking by a qualified professional before real publication.